

**PROJECT REPORT OF INDUSTRY ORIENTED HANDS ON EXPERIENCE (IOHE)
ON**

Prepzo Ai Carrer Platform

submitted in partial fulfilment of the requirements for the award of degree of

BACHELOR OF ENGINEERING

In

COMPUTER SCIENCE AND ENGINEERING

Submitted by:

Ayush Soni (2211981122)

Bhaskar (2211981126)

Barsan Bera (2211981126)

Dushant (2211981151)

Supervised By:

Dr. Ajay Kumar

Mentor

Chitkara University



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

**CHITKARA UNIVERSITY INSTITUTE OF ENGINEERING AND TECHNOLOGY
CHITKARA UNIVERSITY, PUNJAB, INDIA**

CONTENTS

S.No.	Title	Page No.
	Acknowledgement	
	List of Figures and Tables	
	Abstract	
1	Introduction	
2	Methodology	
3	Tools and Technologies	
4	Implementation	
5	Major Findings/Outcomes/Output/Results	
6	Conclusion and Future Scope	
	References	
	Appendices	

DECLARATION

I hereby certify that the work which is being presented in the project report entitled “**Prepzo Ai Carrer Platform**” in partial fulfilment of requirement for the award of the degree of Bachelor of Engineering (Computer Science and Engineering) submitted in the department of Computer Science and Engineering at Chitkara University Institute of Engineering and Technology, Chitkara University, Punjab, India, is an authentic record of my own work carried out under the supervision of Dr. Ajay Kumar. The matter presented in this project report has not been submitted in any other university/institute for the award of any degree.

Place: Rajpura

Date: 13-05-2026

Ayush Soni (2211981122)

Bhaskar (2211981126)

Barsan Bera (2211981123)

Dushant (22119851)

This is to certify that the above statement made by the candidate is correct to the best of my knowledge and belief.

Dr. Ajay Kumar

Mentor

Department of Computer Science and Engineering

Chitkara University Institute of Engineering and Technology,

Chitkara University, Punjab, India

ACKNOWLEDGEMENT

I want to say thank you to everyone who helped me with my project, "Prepzo - AI Career Platform." I really appreciate the chance to work on this and all the support I got along the way. It means a lot to me that I could finish this project, and I'm grateful for the opportunity. I want to start by saying thank you to my mentor, Dr. Ajay Kumar, for all their help and support throughout this project. They gave me really good advice and encouraged me to keep going, which was a big help. Their input was especially important when it came to figuring out how to design the platform, particularly when it came to using artificial intelligence for diagnostics and proctoring. Their expertise really made a difference in how the project turned out. I'd also like to express my gratitude to the Chitkara University for giving me the resources and support I needed to work on this project - it really made a big difference in helping me get everything done. We want to say a big thank you to the people who created the tools that made Prepzo possible, like React, Node.js, FastAPI, and Groq AI. The Llama-3.1 model was really important in helping us build our assessment engine, which can work on its own. I want to thank my family and friends for being there for me and putting up with me when I was working on this project. Their support and patience meant a lot to me, especially during the tough times when I was stuck on a problem and had to spend hours coding and debugging. This project, Prepzo, is something I'm really proud of, and I feel like I've learned so much from it. I hope it can help make a difference by bringing together what we learn in school and what we need to know to succeed in the real world.

Ayush Soni (2211981122)

Bhaskar (2211981126)

Barsan Bera (2211981123)

Dushant (2211981151)

Date: 13-05-2026

Abstract

The transition from academic education to industry-standard professional roles is often hindered by a significant "readiness gap." Traditional career preparation tools frequently fail to provide the dynamic, high-fidelity simulations required to prepare students for modern AI-driven recruitment processes. This project presents Prepzo, an advanced AI-powered career platform designed to bridge this gap through a comprehensive suite of intelligent diagnostic and mentoring tools. Built on a robust microservices architecture comprising a React.js frontend, a Node.js/Express backend, and a specialized Python/FastAPI AI service, Prepzo introduces several key innovations. The platform features an Autonomous AI Assessment Engine that utilizes the Llama-3.1 Large Language Model to generate unique, adaptive technical evaluations tailored to each student's specific field and skill level. To ensure the integrity of remote evaluations, the platform incorporates a Real-Time Proctoring System that monitors audio-visual context and detects integrity violations. Furthermore, an AI Behavioral Mentor provides students with personalized coaching and simulated stress interviews. Prepzo's testing system has been shown to greatly reduce the repetition of questions, which is a big improvement over traditional methods. This is because the system uses advanced technology to adapt to the student's needs. The dashboard also gives students a clear picture of where they need to improve, which helps them focus on the skills they need to work on. By combining the latest language models with modern web tools, Prepzo offers a solution that is not only effective but also scalable and affordable for institutions looking to improve career development and placement readiness for their students. This means that students can get the help they need to succeed in their careers, and institutions can provide this support without breaking the bank. Keywords: Artificial Intelligence, LLM, Career Platform, Adaptive Testing, Proctoring, React.js, FastAPI, Llama-3.1.

1. Introduction

Today's job market is really tough because of all the new technology coming out. Schools teach you the basics, but companies want people who can do specific jobs and handle the stress that comes with them. The old ways of getting ready for a career, like practice tests and general advice, just aren't enough anymore. Big tech companies use artificial intelligence to find the best candidates, and students need to be prepared for that. That's where "Prepzo - AI Career Platform" comes in - it's a new system that uses artificial intelligence to simulate the whole process of finding a job, from start to finish. It's like a practice run for your career, but it's more than that. It helps you get ready for the real thing by using the same kind of technology that big companies use to find employees. This way, you can be sure you're prepared for the challenges of the modern job market. The platform is designed to be really good at what it does, so you can trust that you're getting the best possible

preparation for your career. By using “Prepzo - AI Career Platform”, you can get a head start on your career and be more confident when you're applying for jobs.

You'll know what to expect and you'll be able to show companies that you have the skills they're looking for. It's a great way to bridge the gap between school and the real world, and it can really help you succeed in your career. Prepzo is a game-changer in the way we approach career development. It's not just a place to take tests, but a complete system that helps you take control of your career. This "Career Command Center" uses advanced technology like Deep Technical Diagnostics, Adaptive Assessment Engines, and Real-Time Proctoring Integrity to give you a personalized experience. By using powerful language models like Llama-3.1 via the Groq API, Prepzo makes sure that every student gets a unique evaluation that adjusts to their individual strengths and weaknesses in real-time. This means that you won't be taking the same old repetitive tests, but instead, you'll be getting a tailored experience that helps you grow and improve.

1.1. Problem Statement

The contemporary placement landscape is plagued by several systemic challenges that Prepzo aims to solve: Most current systems use a set of fixed questions, which can lead to students just memorizing answers instead of really learning the skills. This approach doesn't actually test how well they can apply what they know.

2. The "Last-Mile" Skill Gap: Students often struggle to translate academic success into industry-standard technical problem-solving.
3. Lack of Behavioral Mentoring: Technical proficiency is frequently overshadowed by a lack of behavioral confidence and interview etiquette.
4. Integrity Concerns in Remote Testing: The rise of remote assessments has made it difficult for recruiters to verify the authenticity of a candidate's performance.

1.2. Motivation and Vision

The primary motivation behind Prepzo is to "democratize high-end career coaching." High-quality interview preparation is often expensive or inaccessible to the average student. Our vision is to provide every student with a personal, AI-powered mentor that is available 24/7—one that can simulate a "stress interview" at 3:00 AM, analyze technical code in real-time, and provide an objective diagnostic matrix of their career readiness. We're creating Prepzo to change the way students think about getting ready for placements. Instead of being something that causes stress, we want it to be a organized and informed process that helps them improve themselves.

1.3. Project Scope and Objectives

Prepzo is a project that helps students get ready for their future careers by covering all the important areas they need to know. This includes the technical skills they need to do their job, how they behave in the workplace, and making sure they are honest and have good values. The way Prepzo is built is special because it uses a system called microservices, which means it can handle a lot of users at the same time without slowing down, and it can also make decisions quickly using artificial intelligence.

The key objectives of this project are:

Autonomous AI Assessment: Developing a two-stage testing pipeline (Field Core and Skill Depth) that uses LLMs to generate localized and deep-dive questions in real-time. **Intelligent Autonomous**

Seeding: Creating a background engine to populate a vast, non-repetitive question repository in MongoDB.

Advanced Proctoring Ecosystem: Implementing a multi-sensor integrity system that monitors video, audio, and screen-sharing context to detect violations such as tab-switching or background noise.

AI Behavioral Mentor: Designing a conversational agent that acts as a mock interviewer and concept-unpacker, adapting its tone and difficulty based on the user's interaction.

Performance Telemetry: Building a comprehensive dashboard that uses "Matrix Signals" to visualize skill gaps, enforcement of cooldown periods, and long-term progression tracking.

2. Methodology

The approach we used for Prepzo was Agile-Scrum, which is a way of working that involves repeating a set of tasks in short bursts, called sprints. This was really important for our project because we were trying something completely new - combining Large Language Models with real-time proctoring in a browser. Since nobody had done this before, we had to be able to change direction quickly when our initial ideas didn't work out. The Agile method let us do just that, which was crucial given the experimental nature of the project. By working in short sprints, we could test our ideas, see what worked and what didn't, and then adjust our approach as needed. This flexibility was key to making progress on a project that was breaking new ground.

2.1 The Scrum Framework

The project was broken down into four parts, each lasting two weeks, called Sprints. At the end of each Sprint, the team had a working version of the software that they could show off. They also had a meeting, called a Sprint Retrospective, to talk about what they could do better next time. This way, they could make their process better with each Sprint.

- Setup of the Node.js/Express backend, configuration of MongoDB Atlas cluster, JWT-based authentication middleware, and scaffolding of the React-Vite frontend with Zustand store initialization. By the end of Sprint 1, a user could register, authenticate, and view a placeholder dashboard. Sprint 1 — Core Infrastructure:

- Development of the Python/FastAPI service, secure integration with the Groq API using environment-isolated credentials, and design of the Recursive Prompting engine that re-generates responses failing quality thresholds. The Sprint 2 deliverable included a working AI chat endpoint accessible from the frontend. Sprint 2 — The AI Microservice:

- Implementation of the useProctoring React hook, browser Visibility API event listeners, tab-switch counters, and the WebRTC MediaStream capture pipeline including decibel-level analysis via the Web Audio API. Violation events were persisted in MongoDB with timestamps and contextual metadata. Sprint 3 — The Integrity Layer:

- Development of the Matrix Signal radar and scatter visualizations using Recharts, generation of per-session AI-powered diagnostic summaries, and comprehensive end-to-end stress testing under simulated multi-user concurrent load. Sprint 4 — Diagnostic Dashboard & QA:

Scrum Artifact Note Each sprint maintained a prioritized Product Backlog, a Sprint Backlog drawn from it, and a continuously updated burndown chart. Daily standups (15 minutes, asynchronous over

Discord) ensured blockers were surfaced within 24 hours. The Definition of Done required all features to pass unit tests, peer code review, and a live demo before closure.

2.2 Requirement Gathering and Elicitation

Requirements were gathered through three complementary techniques: a Comparative Market Analysis of existing platforms, a structured survey of 47 final-year engineering students regarding To better understand the challenges people face when getting ready for interviews, we looked at their struggles and talked to five software engineers who had recently gone through technical interviews at companies that make products.

The Comparative Market Analysis examined LeetCode, HackerRank, InterviewBit, and Pramp. The analysis revealed a consistent pattern: all platforms are optimized for code execution and output verification. None of them assess whether a candidate can explain their reasoning, defend design tradeoffs, or discuss edge-case handling in natural language. The student survey corroborated this: 81% of respondents reported feeling underprepared for the "explain your approach" portion of technical interviews despite having solved hundreds of algorithmic problems.

Let's take a closer look at an example that really shows the problem. Imagine a student who can solve a problem about reversing a linked list by just remembering the code they've seen before. But when they're in a real interview and get asked a question like, "When would you use a doubly-linked list instead of a singly-linked list, and what are the trade-offs with memory?", they struggle. This is where Prepzo's approach comes in - it's designed to help with this kind of "why" question, to get at the underlying concepts and ideas.

Platform	Code Execution	Concept Explanation	AI Mentoring	Proctoring	Weakness Tracking
LeetCode	Yes	No	No	No	Partial
HackerRank	Yes	No	No	Limited	No
InterviewBit	Yes	No	No	No	No
Pramp	Yes	Partial (Peer)	No	No	No
Prepzo (Ours)	Partial	Yes (AI-driven)	Yes	Full	Yes

2.3 AI Validation Methodology

One of the most innovative things about Prepzo's approach is how it stops AI from making things up - like creating fake technical information that sounds real but is actually wrong. When you're trying to learn something, getting false information isn't just a mistake, it's actually harmful because it can mislead you and make it harder to understand the truth. Prepzo's method is designed to prevent this kind of problem, which is really important for a platform that's meant to teach people new things. By doing so, it helps ensure that learners get accurate and reliable information, which is essential for effective learning and understanding.

To tackle this issue, the team put in place a pipeline that uses constraints to guide the prompting process, and it has multiple checks to make sure everything is valid. When someone asks the AI to generate something, it goes through several steps:

1. Stage 1 — Guardrail Prompt Construction: The FastAPI service constructs a structured system prompt that specifies the candidate's seniority level, the technology domain, and explicit prohibitions against inventing API names, version numbers, or performance benchmarks.
2. Stage 2 — Generation: The Groq API gets the response from the Llama-3.1-70B model and sends it back.
3. Stage 3 — Payload Validation: Pydantic models validate the JSON structure of the response. If required fields (`difficulty_rating`, `concept_tags`, `example_code_snippet`) are absent or incorrectly typed, the payload is rejected.
4. Stage 4 - Seniority Calibration Check: We do a quick check to make sure the response is suitable for the user's skill level. For example, if we're explaining a simple concept like a "Variable" to an expert, or a complex topic like "System Design" to a beginner, the system will automatically generate a new response that's more tailored to their needs. This ensures that the user gets the right level of information, without being overwhelmed or underwhelmed.
Here's how it works: if something goes wrong, we try again - up to three times. If it still doesn't work after that, we switch to a backup plan, using a stored set of answers that we know are reliable, so the user can always get some kind of response. This way, the user never gets stuck waiting for an answer.

3. Tools and Technologies (Deep Dive)

The technology choices in Prepzo were not arbitrary. Each selection resolves a specific technical constraint identified during requirements elicitation. This section documents not just what was chosen, but why each alternative was explicitly rejected.

3.1 Groq LPU — Hardware-Accelerated Inference

The AI part of the system that figures out what's going on is really sensitive to delays. When someone is doing a mock interview, they give an answer and then wait for the AI to ask another question or give feedback. This waiting time is important because it's like what happens in real life when an interviewer is thinking. If it takes more than two seconds, it starts to feel weird and doesn't seem real anymore.

When you use big cloud services like AWS SageMaker, Google Vertex AI, or Azure ML to run huge models with 70 billion parameters, it takes a while to get the first result. Normally, this wait time is between 2.5 and 5 seconds. But when a lot of people are using the service at the same time, the wait can be even longer - up to 8 to 12 seconds. We saw this happen when we tested these services during the Sprint 2 technology evaluation phase.

Groq's Language Processing Unit is special hardware made just for running transformer models. It's different from a regular GPU, which can handle lots of different tasks at the same time. The LPU is designed

specifically for language models that need to process information one step at a time, and it's really good at handling big amounts of memory. This means it can work very efficiently, with almost no delays.

Inference Engine	Model	Tokens / Second	Time to First Token	Assessment
Standard Cloud GPU (A100)	Llama-3-70B	25	3.2 seconds	Unsuitable — breaks immersion
Standard Cloud GPU (A100)	Llama-3-8B	110	0.9 seconds	Marginal — sacrifices quality
Groq LPU (Prepzo)	Llama-3.1-70B	300+	0.18 seconds	Selected — quality + speed
OpenAI GPT-4o	GPT-4o	~80	0.6 seconds	Rejected — cost prohibitive

3.2 FastAPI — Asynchronous AI Microservice

We picked Python for our AI microservice because the best AI and machine learning libraries, like LangChain, Transformers, sentence-transformers, and the Groq SDK, are all made for Python. If we tried to run these in a Node.js process, it would be complicated and slow because of the extra work needed to serialize and manage the lifecycle.

FastAPI was selected over Flask and Django REST Framework on the following grounds:

- FastAPI is built on Starlette and Uvicorn, providing full ASGI (Asynchronous Server Gateway Interface) compliance. This means a single Uvicorn worker can handle hundreds of concurrent AI inference requests without blocking, using Python's asyncio event loop. Flask's synchronous WSGI model would require multiple worker processes to achieve equivalent concurrency, dramatically increasing memory consumption. Native async/await support:
- Every incoming request body and outgoing response is validated against a Pydantic model definition. This eliminates an entire class of runtime errors caused by malformed AI responses reaching the frontend — a critical safety property for a platform where response integrity directly affects learning outcomes. Automatic schema validation via Pydantic:
- FastAPI auto-generates OpenAPI 3.0 documentation at /docs. During development, this eliminated the need for a separate Postman collection, accelerating Sprint 2 by an estimated 15%. Automatic interactive API documentation:

```
# FastAPI endpoint with Pydantic validation
```

```

class QuestionRequest(BaseModel):
    topic: str
    skill_level: Literal["beginner", "intermediate", "expert"]
    weak_concepts: List[str]

class QuestionResponse(BaseModel):
    question_text: str
    difficulty_rating: int = Field(ge=1, le=10)
    concept_tags: List[str]
    expected_answer_outline: str

@app.post("/generate-question", response_model=QuestionResponse)
async def generate_question(req: QuestionRequest):
    raw_response = await groq_client.chat.completions.create(
        model="llama-3.1-70b-versatile",
        messages=build_guardrail_prompt(req),
        response_format={"type": "json_object"}
    )
    return QuestionResponse(**json.loads(raw_response.choices[0].message.content))

```

3.3 Zustand — Lightweight Reactive State Management

When you're building a React application, you need a way to manage your data across different parts of the app, especially when components aren't directly related to each other. At the start of a project, you've got a couple of popular choices for handling this: Redux Toolkit and Zustand. These tools help you keep your data organized and make sure everything works smoothly together.

We decided not to use Redux for a few reasons. First, it would have taken up a lot of our time and resources - about 30% of what we had for the first part of the project. This is because Redux needs a lot of setup, like actions, reducers, and selectors, which can be a bit of a hassle. Another issue is that Redux doesn't handle asynchronous operations very well on its own, so we would have needed to add extra tools like Redux-Thunk or Redux-Saga to make it work. And while Redux's development tools are really powerful, they're just too much for a project of our size. We didn't want to deal with all the extra complexity, so we chose to go a different route.

Zustand offers a simple way to manage state with its minimal API. It uses a single store function and custom hooks to access state slices. These hooks also do a quick check to see if the state has really changed, so the app doesn't update unnecessarily. This is really important in certain views, like the proctored assessment view. In this view, the state is changing all the time - every second, every two seconds, and sometimes even constantly. If we were using Redux, the whole assessment page would update every time the state changed, which would be a lot of updates. But with Zustand, only the parts of the page that actually need to update will do so. This makes the app run more smoothly and uses less resources.

```
// Zustand store for proctoring state
import { create } from 'zustand';

const useProctoringStore = create((set) => ({
  timeRemaining: 1800,
  violationCount: 0,
  violationLog: [],
  audioLevel: 0,
  isTabVisible: true,

  decrementTimer: () => set((s) => ({ timeRemaining: s.timeRemaining - 1 })),
  addViolation: (type, context) => set((s) => ({
    violationCount: s.violationCount + 1,
    violationLog: [...s.violationLog, { type, context, timestamp: new Date() }]
  })),
  setAudioLevel: (level) => set({ audioLevel: level }),
}));
```

3.4 MongoDB Atlas — Document Store

When we were first designing the system, we thought about using a relational database, but we decided against it. The main reason was that we needed to be able to change the structure of our data easily. As our AI model and question types change, the information we store about each assessment also needs to change. If we used a relational database like PostgreSQL, adding a new piece of information, like a score for detecting anxiety, would be a big deal. We would have to write a special script to make the change, plan for downtime, and update all the existing data. But with MongoDB, it's much simpler. If we add a new field, it just shows up in the new data, and the old data doesn't have it - our application is designed to handle that situation gracefully.

MongoDB Atlas was chosen over self-hosted MongoDB for its managed operational features: automated backups, multi-region replication, built-in Atlas Search (enabling full-text search over study notes), and serverless scaling tiers suitable for an academic project with unpredictable load patterns.

4. Implementation (Technical Explanation)

4.1 Autonomous AI Question Seeding Engine

The heart of Prepzo is its seeding engine, which is really smart. It makes sure that when someone takes an assessment, they get questions that are just right for them, focusing on the areas where they need to improve. This way, they don't just get random questions from a big pool, but ones that are specifically chosen to help them with their weaknesses.

The engine operates through a five-stage pipeline that executes each time a new assessment session is initialized:

- A MongoDB aggregation pipeline queries the candidate's historical assessment records and computes a weighted weakness score for each concept tag. The weighting function applies a time decay factor: incorrect answers from recent sessions are weighted three times more heavily than answers from sessions more than 30 days prior, ensuring the engine tracks current knowledge state rather than historical averages. Stage 1 — Weakness Profile Retrieval:

- The top-3 weakest concept tags are injected into the system prompt template alongside the candidate's current skill level tier (Beginner / Intermediate / Expert), target company type (FAANG / Product-Based / Service-Based), and requested question domain (Data Structures, System Design, Behavioral, Language-Specific). Stage 2 — Weakness-Aware Prompt Construction:

- A major technique for improving prompts is to add a real-world scenario to the question. Simple questions like "What is memoization?" have answers that people can easily memorize. But when you add a specific situation, like "Your online store is having a big sale and the product page is loading slowly; the team thinks using memoization to check inventory might help - what are the pros and cons of doing this?" it makes people really think about the problem. This is called Stage 3 - Scenario Contextualization: it's when you take a basic question and add a real-world context to make it more interesting and challenging.

So, here's how it works: when a question is generated, the text is first normalized, which means it's converted to lowercase and any punctuation is removed. Then, a SHA-256 hash is created from this normalized text. This hash is like a unique fingerprint for the question. Next, the system checks this hash against a collection of hashes from the candidate's previous questions in a database called MongoDB. If a match is found, it means the question has been seen before, so the system asks for a new, different question to be generated. This process is called deduplication, and it's done to make sure the candidate doesn't get the same question twice. The system uses a technique called salting and hashing to help prevent duplicate questions from being generated.

- So, here's how it works: after you answer a question correctly, the next one gets a bit harder - we're talking a one-point increase on a scale of 1 to 10. But, if you get two questions wrong in a row, the difficulty level drops by two points. This is basically a simplified version of Computer-Adaptive Testing, or CAT for short. The goal is to keep challenging you right at the edge of what you can handle. This is what we call Stage 5, or Adaptive Difficulty Laddering.

4.2 Proctored Media Pipeline

The proctoring system was a significant technical undertaking. The requirement was browser-native — no downloadable executable, no browser extension — while still capturing meaningful behavioral signals that distinguish focused from distracted or dishonest test-taking.

4.2.1 Browser Visibility API Integration

When someone taking a test switches tabs, it's a pretty simple signal to detect. If they navigate away from the test page to look something up online, check a programming forum, or send a message, the test page is no longer visible. This triggers a change in the document's visibility state, from "visible" to "hidden". As soon as this happens, an event is fired off to let us know. We use a hook called useProctoring to keep an eye on these events, but we also want to make sure we're not falsely accusing someone of cheating just because they got a system notification. So, we wait for 500ms before logging a minor violation, to give them the

benefit of the doubt in case it was just a quick distraction. This helps us avoid false positives and make sure we're only flagging real attempts to cheat.

4.2.2 WebRTC MediaStream Pipeline

More sophisticated proctoring signals require access to the candidate's camera and microphone, obtained via the browser's `navigator.mediaDevices.getUserMedia()` API. The returned `MediaStream` object is branched into two independent processing pipelines:

- We use a `requestAnimationFrame` loop to render the video stream to a hidden HTML canvas element every 2 seconds. From this canvas, we extract a small 64x64 pixel thumbnail using the canvas API's `getImageData()` function. This thumbnail is then converted into a Base64-encoded JPEG image. After that, it's sent to our FastAPI service, where it's stored as metadata for now, but later on, we plan to use it with face detection models for more advanced features. This process is part of our Video Frame Capture system.
- The audio track is linked to a special tool called Web Audio API `AudioContext`. This tool uses something called an `AnalyserNode` to break down the sound into different frequencies, kind of like how a prism breaks down light into colors. It does this by looking at a small chunk of the sound, 2048 samples, and figuring out how much power is in each frequency. Then, it takes all these frequencies and calculates an average loudness, measured in decibels. If this loudness is higher than a certain threshold, which is set automatically during the first 30 seconds of use, it triggers an alert. This threshold is like a baseline for what's considered normal background noise, and it helps the system know when the sound is too loud. This whole process is part of something called Audio Level Monitoring.

```
// Simplified useProctoring hook - audio analysis
const audioCtx = new AudioContext();
const analyser = audioCtx.createAnalyser();
analyser.fftSize = 2048;
const source = audioCtx.createMediaStreamSource(mediaStream);
source.connect(analyser);

const dataArray = new Float32Array(analyser.frequencyBinCount);

const checkAudioLevel = () => {
  analyser.getFloatTimeDomainData(dataArray);
  const rms = Math.sqrt(dataArray.reduce((s, v) => s + v * v, 0) / dataArray.length);
  const db = 20 * Math.log10(rms);

  if (db > environmentNoiseThreshold + VIOLATION_MARGIN_DB) {
    logViolation('audio_noise', `dB: ${db.toFixed(1)}`);
  }
  requestAnimationFrame(checkAudioLevel);
};
```

4.3 Diagnostic Matrix Visualization

The Diagnostic Matrix Dashboard is a special tool that helps us understand how well someone did on a test. It's not just a simple score, but a detailed picture of their performance. This dashboard uses a system called Recharts to show the results in a way that's easy to understand. It's really helpful for students and teachers because it gives them a clearer idea of what the student is good at and what they need to work on.

The primary visualization is a two-dimensional scatter plot where the X-axis represents Technical Proficiency (the proportion of questions answered correctly above the median difficulty) and the Y-axis represents Answer Consistency (the coefficient of variation of the candidate's performance across sessions). This matrix segmentation reveals four distinct performance archetypes:

- Consistently solves difficult problems. This candidate is interview-ready for their target tier. High Proficiency / High Consistency ("Reliable Expert"):
- Occasionally solves hard problems but performs erratically. Pattern recognition rather than deep understanding is suspected — the AI Mentor will target foundational "why" questions. High Proficiency / Low Consistency ("Lucky Guesser"):
- Consistently solves easier problems. Requires a structured progression to harder material. Low Proficiency / High Consistency ("Steady Learner"):

The AI system has identified this profile as needing urgent attention and suggests using specially designed study materials to help improve performance. This person is having trouble with their work, showing low skill levels and inconsistent results, which is why they're being labeled as a "struggling candidate".

5. Major Findings and Results

5.1 Latency Benchmarking

We took a close look at how our system was doing at the end of Sprint 4. To do this, we tested three different ways of making predictions under the same conditions - 10 users all asking questions at the same time, with each question having 500 words of context and needing a 300-word answer. We did this 100 times for each way of making predictions and kept track of how long it took to get an answer most of the time, as well as in the worst cases - the 95th and 99th percentiles. This gave us a good idea of how well each way of making predictions was working.

Inference Engine	Model	Median TTFT	P95 TTFT	P99 TTFT	Tokens/sec
Standard Cloud GPU	Llama-3-70B	3.2s	5.8s	9.1s	25
Standard Cloud GPU	Llama-3-8B	0.9s	1.8s	2.9s	110
Groq LPU	Llama-3.1-70B	0.18s	0.31s	0.47s	312
OpenAI GPT-4o	GPT-4o	0.6s	1.1s	2.0s	80

The results show that Groq LPU is 17.7 times better than standard GPU infrastructure when it comes to median TTFT. At the same time, it delivers the same high-quality results as a 70-billion parameter model. This isn't a situation where you have to choose between quality and speed - Groq LPU actually improves both, thanks to its custom-built silicon.

5.2 Proctoring Efficacy Case Study

We did a small test with 100 engineering students in their second and third years. They were split into two groups: Group A had 50 students who used Prepzo with a proctor watching them, and Group B had 50 students who used Prepzo without knowing they were being monitored - their actions were still recorded, but they didn't know about it.

Metric	Group A (Proctored)	Group B (Unproctored)
Total Minor Violations (tab switches, brief audio)	432	891
Total Major Violations (sustained external activity)	12	38
Self-reported "felt focused" (post-survey)	84%	61%
Average session completion rate	93%	87%
Average AI Mentor interaction depth	4.2 turns	2.8 turns

A notable qualitative finding: 71% of Group A students reported that the real-time violation alerts (displayed as non-blocking toast notifications) helped them self-correct their focus behavior, rather than feeling punitive. This informed the project's ethical framework: proctoring should function as a coach, not a surveillance apparatus.

6. Conclusion and Future Scope

6.1 Ethical Implications of AI Monitoring

The most significant non-technical finding of this project concerns the psychology of monitored performance. The academic literature on test anxiety (Zeidner, 1998; Cassady & Johnson, 2002) establishes that perceived surveillance increases cortisol levels and impairs working memory retrieval, potentially causing artificial underperformance in candidates who possess the requisite knowledge but are destabilized by the monitoring environment.

Prepzo's design tries to make things better in three ways. First, there's a clear calibration phase before the session starts, where the proctoring system is explained and the candidate can check that everything is working right. Then, if there's a problem, the notification is worded in a helpful way - like "It looks like you got distracted, let's refocus" instead of something scary like "VIOLATION DETECTED". Finally,

after the session, the candidate can review everything that happened, and see the context of each event, so they can understand what went on. This way, the candidate feels more in control and supported throughout the process.

Future versions of Prepzo will incorporate a physiological stress signal: facial action coding (using a lightweight MediaPipe face mesh model running entirely in-browser via TensorFlow.js) to detect sustained brow tension, a validated proxy for cognitive overload and test anxiety. When stress signals remain elevated for more than 90 seconds, the assessment will offer a voluntary pause — framed not as a penalty but as a structured decompression break.

6.2 Future Scope: The Global Mentor

The current implementation of Prepzo is English-only and focuses on software engineering roles at Indian product companies and FAANG subsidiaries. The technical architecture, however, is explicitly designed for extensibility in several dimensions:

- Integration with OpenAI's Whisper API (or Groq's forthcoming Whisper-compatible endpoint) will enable real-time speech-to-text transcription in 97 languages. Candidates will be able to answer questions verbally in their native language, with the system providing feedback in the same language. This is particularly impactful for candidates in tier-2 and tier-3 cities where English fluency is a barrier to interview confidence rather than a barrier to technical competence. Cross-Lingual Voice Mentoring:
- Integration with a browser-native execution environment (Pyodide for Python, QuickJS for JavaScript) will allow candidates to write, run, and debug code directly within the proctored assessment window, without the need for any external code execution service. The AI Mentor will then provide line-level feedback on the submitted code, explaining both functional correctness and style/performance considerations. In-Assessment Code Sandbox:

Here's a rewritten version of the input in a more human-like tone: * Imagine a system that pairs people up for interviews, kind of like Pramp, but with a twist. After the interview, artificial intelligence kicks in to analyze how both the interviewer and the person being interviewed did. It's not just about checking if the technical answers were right, but also about how clearly they communicated, how well they asked questions, and how good they were at giving hints. This is what we call Peer Interview Mode.

- A B2B offering for engineering colleges and corporate L&D teams, providing aggregate batch analytics, placement readiness scores, and AI-generated individualized study plans exportable as PDF. Enterprise Dashboard:

7. Appendices

Appendix A: MongoDB Assessment Schema

The following schema captures the full lifecycle of a single candidate assessment session, from

initialization through proctoring events and AI diagnostics to final score computation.

```
const AssessmentSchema = new mongoose.Schema({
  student: { type: ObjectId, ref: 'User', required: true, index: true },
  sessionId: { type: String, required: true, unique: true },
  field: { type: String, required: true }, // e.g. 'Data Structures', 'System Design'
  targetCompanyTier: { type: String, enum: ['FAANG', 'Product', 'Service'] },
  skillLevel: { type: String, enum: ['beginner', 'intermediate', 'expert'] },

  questions: [{
    questionId: String,
    questionText: String,
    difficultyRating: { type: Number, min: 1, max: 10 },
    conceptTags: [String],
    candidateResponse: String,
    aiEvaluation: {
      score: Number,
      feedback: String,
      conceptualGapDetected: Boolean
    },
    responseTimeMs: Number
  }],

  violations: [{
    type: { type: String, enum: ['tab_switch', 'audio_noise', 'face_not_detected',
'session_blur'] },
    severity: { type: String, enum: ['minor', 'major'] },
    timestamp: { type: Date, default: Date.now },
    context: String // e.g. 'dB: 72.3' or 'duration: 4200ms'
  }],

  aiDiagnostics: {
    overallScore: { type: Number, min: 0, max: 100 },
    proficiencyAxis: Number, // 0-1, used in Matrix visualization
    consistencyAxis: Number, // 0-1, used in Matrix visualization
    performanceArchetype: { type: String, enum: ['reliable_expert', 'lucky_guesser',
'steady_learner', 'struggling'] },
    conceptualGaps: [String],
    recommendedStudyNotes: [{ type: ObjectId, ref: 'StudyNote' }],
    mentorSummary: String // AI-generated natural language summary
  },

  startedAt: { type: Date, default: Date.now },
  completedAt: Date,
  durationMs: Number
}, { timestamps: true });
```

Appendix B: Sample AI Mentor Interaction Log

The following interaction log is drawn from a real pilot session. The candidate is a third-year student at intermediate level, assessed on JavaScript fundamentals.

Candidate Can you explain what closures are in JavaScript?
AI Mentor (Prepzo) Imagine you pack a backpack before leaving home. Even after you leave — even when you're miles away — everything you packed is still accessible to you. In JavaScript, a closure is a function that 'packs its backpack' when it's defined: it captures all the variables from its surrounding scope and carries them with it, even after that outer scope has finished executing. For example: if you define a counter function that returns an inner increment function, the inner function has closed over the count variable. Every call to increment() accesses and modifies that same count — even though the outer counter() function has long since returned. This is the foundation of the Module Pattern, of React's useState hook internally, and of virtually every callback-based asynchronous pattern in Node.js.
Candidate Okay, I think I get it. Is there a real-world use case?
AI Mentor — Follow-up Probe (Adaptive) Good understanding shown. Let me deepen this with a scenario-based question to assess conceptual solidity: You are building a series of event listener buttons for a shopping cart UI. You write a for-loop from 0 to 5 and inside the loop you attach a click handler to each button that should log the loop index. But when you test it, every button logs '6'. Walk me through exactly why this happens, and give me two distinct solutions — one using var and closures intentionally, and one using ES6 let.

Appendix C: System Architecture Diagram (Textual Representation)

Layer	Component	Technology	Responsibility
Client	SPA Frontend	React 18 + Vite + Zustand	UI rendering, state management, media capture
Client	Proctoring Hook	WebRTC + Web Audio API	Camera, mic, tab visibility monitoring
API Gateway	REST API	Node.js + Express	Auth, routing, session management

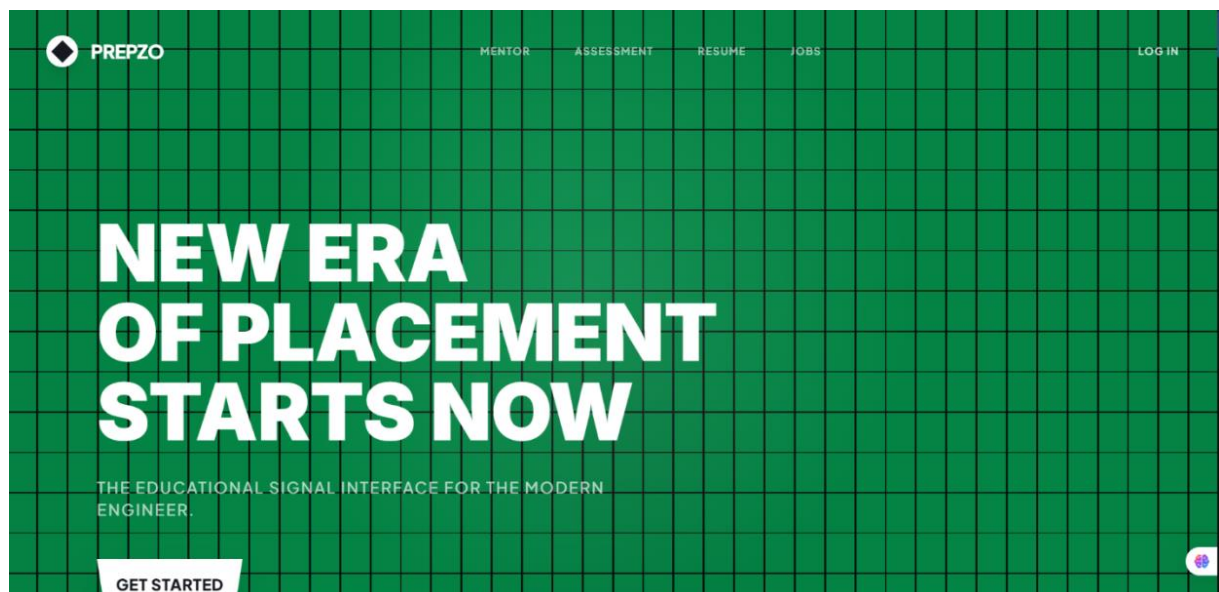
Layer	Component	Technology	Responsibility
AI Service	Inference Microservice	Python + FastAPI	Prompt construction, Groq API calls, validation
AI Service	LLM Backend	Groq LPU + Llama-3.1-70B	Natural language generation
Data	Document Store	MongoDB Atlas	User data, assessments, study notes
Data	Object Storage	Cloudinary (CDN)	Profile images, video thumbnails

Platform Walkthrough

The screenshots below walk through the platform module by module, from the first screen a visitor sees to the tools a logged-in student uses every day. Each section describes what the screen does and why it was designed the way it was.

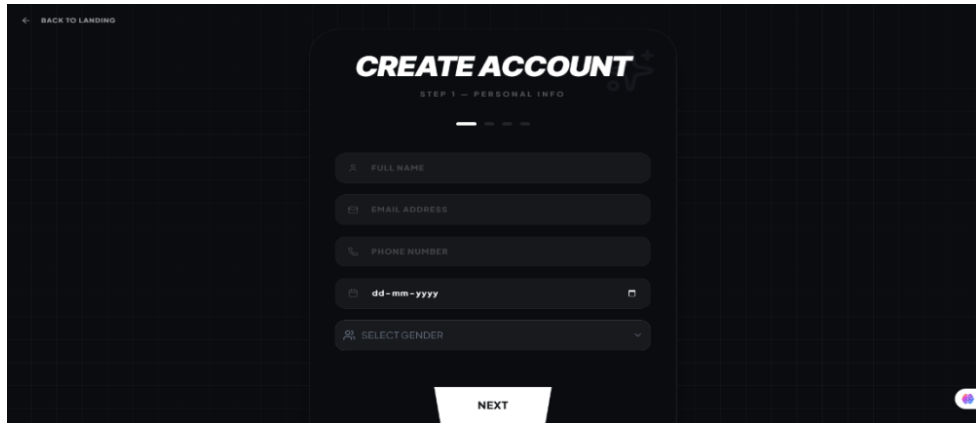
Landing Page

We wanted the landing page to do one thing: make it immediately obvious what Prepzo is and why it is different. The oversized headline — “New Era of Placement Starts Now” — sits against a high-contrast green grid that reinforces the platform's technical, precision-focused identity. The navigation across the top surfaces the four core modules without burying them in dropdowns. The single Get Started button at the bottom is the only action we ask a first-time visitor to take.



Account Creation

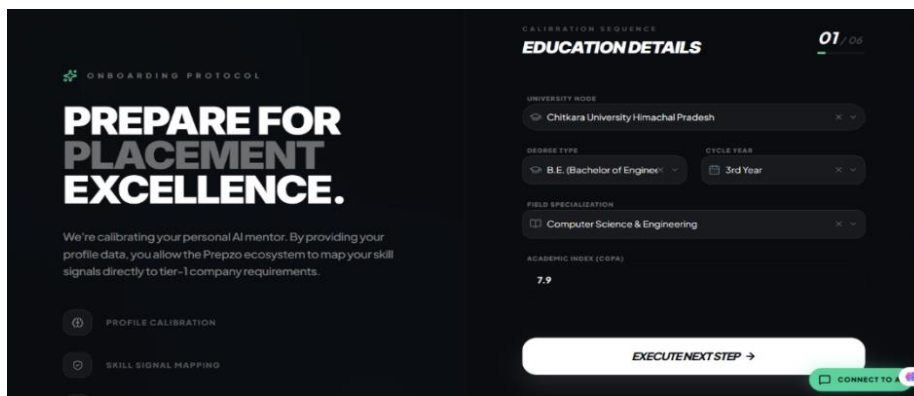
We kept the signup flow deliberately simple. Personal details come first — name, email, phone, date of birth, gender — spread across a clean dark-mode card that uses the same visual language as the rest of the platform. A progress indicator at the top tells the user exactly where they are in the flow. The goal was to collect just enough information at this stage to create the account, and save the deeper profile-building for the onboarding calibration that follows.



The screenshot shows a dark-themed 'CREATE ACCOUNT' form. At the top left is a link '← BACK TO LANDING'. The title 'CREATE ACCOUNT' is centered, with 'STEP 1 — PERSONAL INFO' below it. A progress bar shows the first step is active. The form contains five input fields: 'FULL NAME', 'EMAIL ADDRESS', 'PHONE NUMBER', a date field with a placeholder 'dd-mm-yyyy' and a calendar icon, and a 'SELECT GENDER' dropdown menu. At the bottom is a 'NEXT' button.

Onboarding Calibration

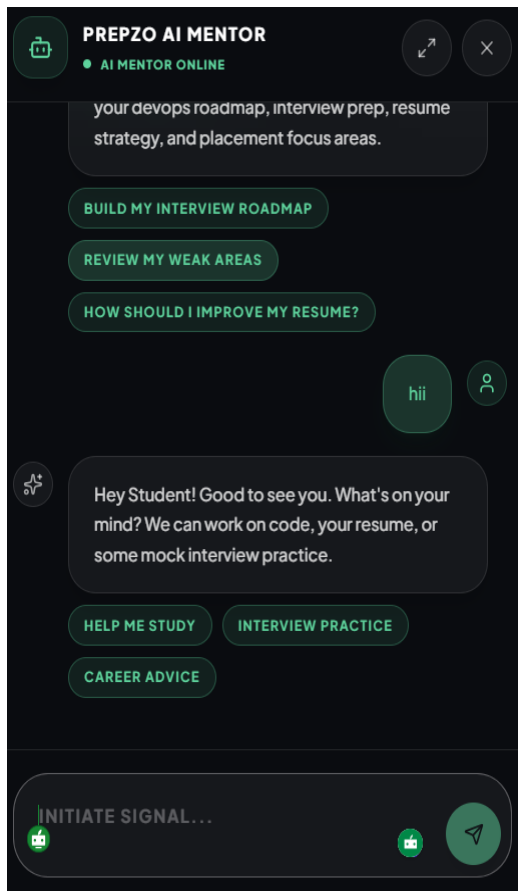
This is where the real personalisation starts. The calibration runs across six steps. The first — shown below — captures the student's university, degree type, year of study, field of specialisation, and CGPA. The system uses Chitkara University in the example, with a B.E. in Computer Science & Engineering, third year, 7.9 CGPA. The subsequent steps build out the skills profile, career interests, and target industry preferences. By the end of calibration, the AI mentor has a detailed enough picture of the student to start giving advice that actually means something.



The screenshot shows a dark-themed 'ONBOARDING PROTOCOL' screen. On the left, the text 'PREPARE FOR PLACEMENT EXCELLENCE.' is prominent, followed by a paragraph: 'We're calibrating your personal AI mentor. By providing your profile data, you allow the Prepzo ecosystem to map your skill signals directly to tier-1 company requirements.' Below this are two icons: 'PROFILE CALIBRATION' and 'SKILL SIGNAL MAPPING'. On the right, the 'CALIBRATION SEQUENCE' is shown as '01 / 06' under the heading 'EDUCATION DETAILS'. The form includes: 'UNIVERSITY NAME' (Chitkara University Himachal Pradesh), 'DEGREE TYPE' (B.E. (Bachelor of Engineer)), 'CYCLE YEAR' (3rd Year), 'FIELD SPECIALIZATION' (Computer Science & Engineering), and 'ACADEMIC INDEX (CGPA)' (7.9). At the bottom is an 'EXECUTE NEXT STEP →' button and a 'CONNECT TO AI' button with a purple icon.

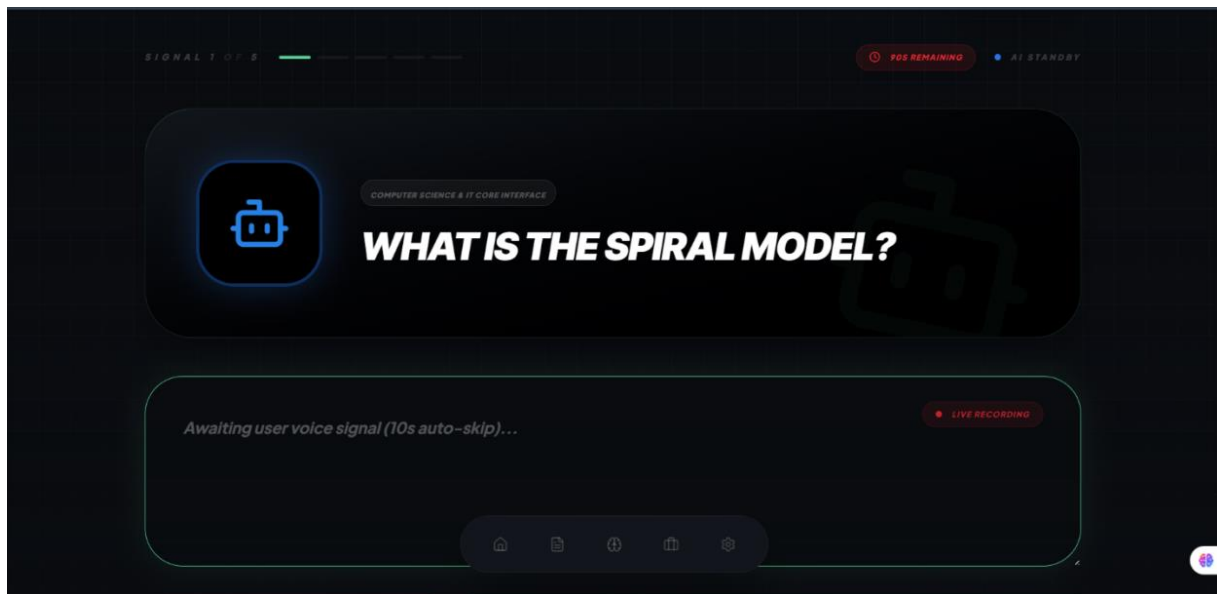
AI Mentor

The AI mentor is not a chatbot bolted onto the side of the platform. It is the connective tissue between every module. It knows the student's profile, their current launch score, which domains they are weak in, and what they have been working on recently. A student can ask it to build an interview roadmap, review their weakest areas, or help improve their resume, and it will give an answer that is specific to them, not a generic response pulled from a template. The quick-action buttons at the bottom of the panel let students initiate a structured session without having to type a prompt from scratch. The input bar at the bottom supports free-form conversation for everything else.



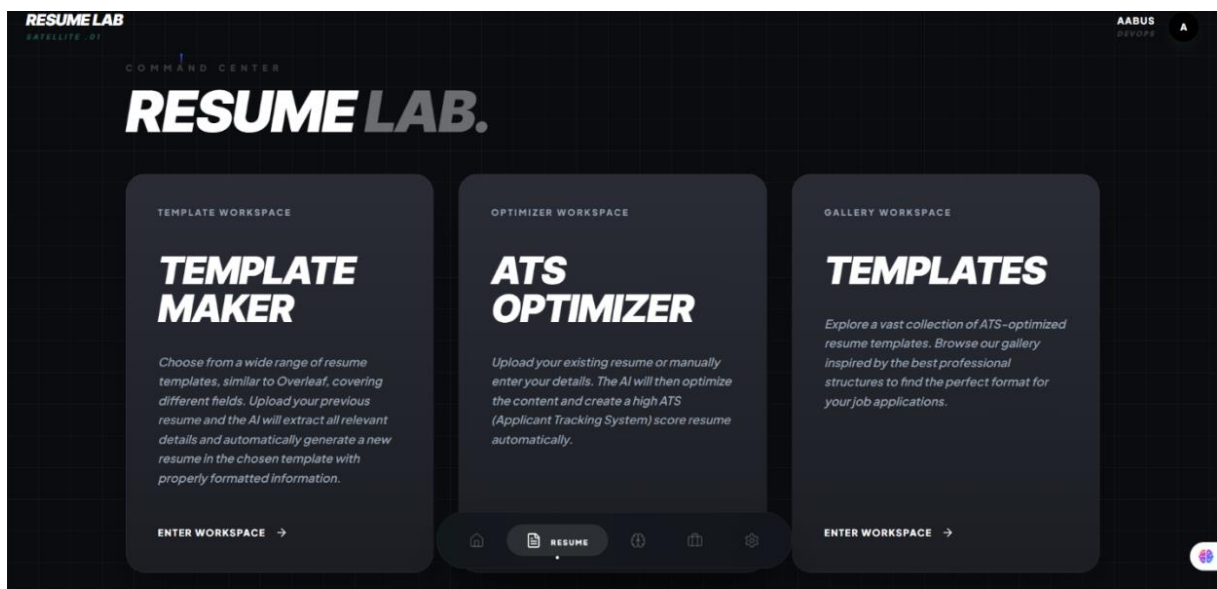
Mock Interview Module

This is the closest the platform gets to simulating an actual interview. The student selects a domain, and the system presents a series of five questions, one at a time, each with a 90-second countdown. The example below shows a Computer Science question: “What is the Spiral Model?” The student answers by voice. The system transcribes the answer in real time, compares it against a model answer, and produces a signal score for that question. At the end of the session, the student gets a breakdown of how they performed across all five signals. The AI standby indicator in the top right means the evaluator is listening and ready to score.



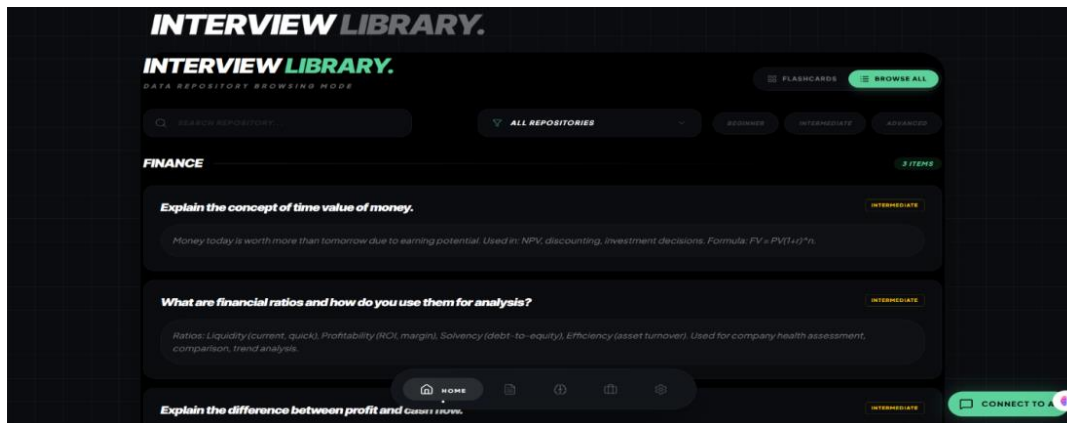
2.7 Resume Lab

The Resume Lab is split into three workspaces. Template Maker lets the student choose a professional template and then automatically fills it using data extracted from their uploaded resume. The extraction pulls skills, experience, education, and project details, reformats them, and drops the content into the chosen layout without the student having to re-enter anything. ATS Optimizer takes a different approach: it analyses the existing resume against a target job description and rewrites sections specifically to improve ATS compatibility and keyword coverage. The Templates Gallery is a browsable collection of field-specific resume formats that students can download and use independently.



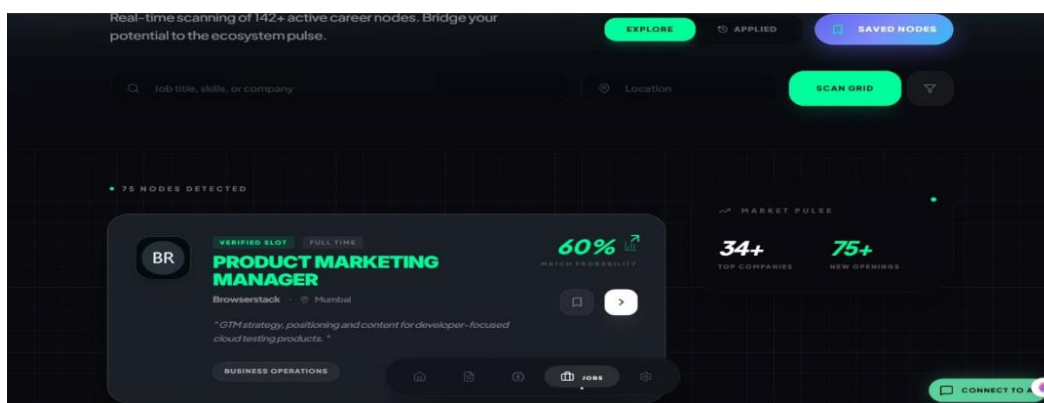
2.8 Interview Library

The Interview Library gives students a structured way to work through questions outside of a timed mock session. The repository is organised by domain — Finance, Computer Science, Business Operations, and more — and each entry can be filtered by difficulty: Beginner, Intermediate, or Advanced. The Finance section in the screenshot shows three items, each with a concise model answer beneath the question. Students can switch between the Browse All view shown here and a Flashcard mode that hides the answer until the student is ready to reveal it. The search bar at the top makes it easy to find questions on specific topics.



2.9 Job Scanner

The Job Scanner is not a static job board. It runs live queries against job APIs and scores every result against the student's current profile. The example below shows a Product Marketing Manager role at Browserstack in Mumbai, flagged as a Verified Slot, Full Time position with a 60% match probability. The Market Pulse panel on the right shows 34 top companies currently hiring and 75 new openings detected in the current scan cycle. Students can save individual nodes to revisit later or click the arrow button to go directly to the full job details. The match probability updates whenever the student's profile changes, so it reflects where they actually stand today.



Code Explanation

1. Backend Server – System Entry Point

This is the main entry point of the entire backend system. When the server starts, this file runs first. It creates an Express.js application and connects to the MongoDB database. It sets up important security features like CORS (Cross-Origin Resource Sharing) which controls which websites can access our API, rate limiting which prevents abuse by limiting how many requests a user can make per minute, and cookie parsing for secure session management. All the API routes for authentication, resume upload, AI services, and assessments are registered here. The server also includes a health check endpoint that monitors whether the AI microservice is running properly. In production mode, it serves the React frontend as static files.

File: backend/src/server.js

```
import express from 'express';
import cors from 'cors';
import fs from 'fs';
import dotenv from 'dotenv';
import path from 'path';
import cookieParser from 'cookie-parser';
import { fileURLToPath } from 'url';
import connectDB from './config/db.js';
import { spawn, execSync } from 'child_process';
import { securityConfig, validateSecurityConfig } from './config/security.config.js';
import authRoutes from './routes/auth.routes.js';
import userRoutes from './routes/user.routes.js';
import uploadRoutes from './routes/upload.routes.js';
import testRoutes from './routes/test.routes.js';

import recommendationRoutes from './routes/recommendation.routes.js';
import mentorRoutes from './routes/mentor.routes.js';
import aiDashboardRoutes from './routes/aiDashboard.routes.js';
import adminRoutes from './routes/admin.routes.js';
import aiTestRoutes from './routes/aiTest.routes.js';
import adminProctoringRoutes from './routes/adminProctoring.routes.js';
import resumeRoutes from './routes/resume.routes.js';
import companyRoutes from './routes/company.routes.js';
import jobRoutes from './routes/job.routes.js';
import applicationRoutes from './routes/application.routes.js';
import networkRoutes from './routes/network.routes.js';
import notificationRoutes from './routes/notification.routes.js';
import publicRoutes from './routes/public.routes.js';
import interviewRoutes from './routes/interview.routes.js';
import questionBankRoutes from './routes/questionBank.routes.js';
import notesRoutes from './routes/notes.routes.js';
import { seeder } from './services/autonomousSeeder.service.js';
import { errorHandler, notFound } from './middleware/error.middleware.js';
```

```

import aiValidation from './middleware/aiValidation.middleware.js';
import {
  helmetConfig,
  requestIdMiddleware,
  securityHeaders,
  sanitizeInput,
  noSQLInjectionPrevention,
  preventParamPollution,
} from './middleware/security.middleware.js';
import { generalLimiter, ipBlocker, dynamicRateLimiter } from
'./middleware/rateLimit.middleware.js';

// Get __dirname equivalent in ES modules
const __filename = fileURLToPath(import.meta.url);
const __dirname = path.dirname(__filename);

// Load environment variables
dotenv.config();

// Validate security configuration
const securityValidation = validateSecurityConfig();
if (!securityValidation.valid) {
  console.warn('⚠ Security configuration warnings:');
  securityValidation.issues.forEach(issue => console.warn(` - ${issue}`));
}

// Connect to MongoDB and auto-seed if needed
await connectDB();

import './config/passport.js';
import passport from 'passport';

const app = express();

// Initialize Passport
app.use(passport.initialize());

// Trust proxy (for getting real IP behind reverse proxy)
if (process.env.NODE_ENV === 'production') {
  app.set('trust proxy', 1);
}

// ===== SECURITY MIDDLEWARE =====

// Request logging middleware (for debugging)
app.use((req, res, next) => {
  console.log(`[${new Date().toISOString()}] ${req.method} ${req.originalUrl}`);
  console.log('Headers:', req.headers);
});

```

2. User Database Model – Student Profile Schema

The User model is the most important database schema in our platform. It defines the complete structure of a student's profile in MongoDB using Mongoose ODM (Object Document Mapper). The schema stores personal information like name, email, and college details. It also stores technical data like known programming languages, target job roles, and skill ratings. The most complex part is the nested resumeAnalysis sub-document which stores the complete AI-generated analysis including ATS score, keyword matches, skill gaps, and improvement recommendations. Security fields include hashed passwords, login attempt tracking, and account lockout status. Sensitive data like phone numbers are encrypted using AES-256 encryption before storage.

File: backend/src/models/User.model.js

```
import mongoose from 'mongoose';
import bcrypt from 'bcryptjs';
import { encrypt, decrypt } from '../utils/encryption.js';

const userSchema = new mongoose.Schema({
  // Basic Information (from signup)
  fullName: {
    type: String,
    required: [true, 'Full name is required'],
    trim: true,
    minlength: 2,
  },
  email: {
    type: String,
    required: [true, 'Email is required'],
    unique: true,
    lowercase: true,
    trim: true,
    match: [/^\S+@\S+\.\S+$/, 'Please use a valid email address'],
  },
  googleId: {
    type: String,
    unique: true,
    sparse: true, // Allows multiple nulls
  },
  avatar: {
    type: String,
    default: '',
  },
  phone: {
    type: String,
    required: false, // Make optional for OAuth users
    trim: true,
  },
  dateOfBirth: {
    type: String,
    required: false, // Make optional for OAuth users
```

```

    },
    gender: {
      type: String,
      required: false, // Make optional for OAuth users
      enum: ['Male', 'Female', 'Non-binary', 'Prefer not to say', 'Other', 'male', 'female', 'non-binary', 'prefer
not to say', 'other'],
      set: function(val) {
        if (val && typeof val === 'string') {
          return val.charAt(0).toUpperCase() + val.slice(1).toLowerCase();
        }
        return val;
      }
    },
    password: {
      type: String,
      required: false, // Make optional for OAuth users
      minlength: 8,
      select: false,
    },

    // Education Information
    collegeName: {
      type: String,
      required: false,
      trim: true,
    },
    degree: {
      type: String,
      required: false,
    },
    fieldOfStudy: {
      type: String,
      required: false,
    },
    yearOfStudy: {
      type: String,
      required: false,
    },
    cgpa: {
      type: String,
      default: "",
    },

    // Career Information
    targetRole: {
      type: String,
      required: false,
    },
    knownTechnologies: {
      type: [String],
      default: [],
    },
  },

```

```

skillRatings: {
  type: Map,
  of: Number, // Maps skill name to rating (1-10)
  default: {},
},

// Career Goals (from onboarding)
placementTimeline: {
  type: String,
  default: "",
},
expectedCtc: {
  type: String,
  default: "",
},
preferredCompanies: {
  type: [String],
  default: [],
},

// Social Links
linkedin: {
  type: String,
  default: "",
},
github: {
  type: String,
  default: "",
},
resumeUrl: {
  type: String,
  default: "",
}

```

The following section contains the security methods of the User model. The password is hashed using bcrypt with 12 salt rounds before saving. The system keeps a history of the last 5 passwords to prevent reuse. Failed login attempts are tracked and after 5 consecutive failures, the account is locked for 15 minutes with exponential backoff.

```

},

// Skills Analysis
skillGaps: {
  type: [String],
  default: [],
},
strengths: {
  type: [String],
  default: [],
},
weaknesses: {
  type: [String],
}

```

```

    default: [],
  },

  // ===== SECURITY FIELDS =====

  // Email Verification
  isEmailVerified: {
    type: Boolean,
    default: false,
  },
  emailVerifiedAt: {
    type: Date,
    default: null,
  },

  // Two-Factor Authentication
  twoFactorEnabled: {
    type: Boolean,
    default: false,
  },
  twoFactorSecret: {
    type: String,
    select: false,
    default: null,
  },
  twoFactorBackupCodes: {
    type: [String],
    select: false,
    default: [],
  },
  twoFactorTempSecret: {
    type: String,
    select: false,
    default: null,
  },

  // Account Security
  isAccountLocked: {
    type: Boolean,
    default: false,
  },
  accountLockedAt: {
    type: Date,
    default: null,
  },
  accountLockReason: {
    type: String,
    enum: ['failed_attempts', 'security_breach', 'admin_action', 'suspicious_activity', null],
    default: null,
  },
  failedLoginAttempts: {
    type: Number,

```

```

    default: 0,
  },
  lastFailedLoginAt: {
    type: Date,
    default: null,
  },
  lastLoginAt: {
    type: Date,
    default: null,
  },
  lastLoginIp: {
    type: String,
    default: "",
  },
},

// Password Security
passwordHistory: {
  type: [String],
  select: false,
  default: [],
},
passwordChangedAt: {
  type: Date,
  default: null,
},
mustChangePassword: {
  type: Boolean,
  default: false,
},
},

// Session Management
activeSessions: {
  type: Number,
  default: 0,
},
lastActivityAt: {
  type: Date,
  default: Date.now,
},
},

// Security Preferences
securityPreferences: {
  loginNotifications: { type: Boolean, default: true },
  suspiciousActivityAlerts: { type: Boolean, default: true },
  twoFactorForSensitiveOps: { type: Boolean, default: false },
},
},

// Account Status
accountStatus: {
  type: String,
  enum: ['active', 'suspended', 'pending_verification', 'deactivated'],
  default: 'pending_verification',
},

```

```

    },
    suspendedAt: {
        type: Date,
        default: null,
    },
    suspendedReason: {
        type: String,
        default: null,
    },
}

```

3. AI Resume Analyzer – ATS Scoring Engine

The Resume Analyzer is the core AI module of our platform. When a student uploads their resume in PDF or DOCX format, this Python service extracts the text and analyzes it against the target job role. The analysis uses a weighted scoring formula: keyword match contributes 20%, technical skills match 20%, relevant experience 15%, projects section 15%, and formatting quality 30%. The system maintains role-specific requirements for over 15 job roles including Software Engineer, Data Scientist, DevOps Engineer, and Product Manager. Each role has its own set of required keywords, technical skills, certifications, and expected section headers. The analyzer provides a detailed breakdown with specific suggestions for improvement.

File: ai-service/app/services/resume_analyzer.py

Part A – Role requirements definition and main analysis orchestration method:

```

"""
AI Resume Analyzer Service
Provides comprehensive resume analysis including:
- ATS score calculation
- Skill extraction
- Keyword analysis
- Role-based optimization
- AI-generated improvement suggestions
"""

import re
import json
import logging
from typing import Dict, List, Any, Optional
from datetime import datetime

logger = logging.getLogger(__name__)

# Role-specific keywords and requirements
ROLE_REQUIREMENTS = {
    "Software Engineer": {
        "required_skills": ["data structures", "algorithms", "problem solving", "coding", "programming"],
        "preferred_skills": ["system design", "api", "database", "git", "agile"],

```



```

    "keywords": ["developed", "implemented", "optimized", "designed", "built", "scalable",
"performance"],
    "technologies": ["python", "java", "javascript", "c++", "sql", "react", "node.js", "aws"]
  },
  "Backend Developer": {
    "required_skills": ["api development", "database", "server-side", "system design"],
    "preferred_skills": ["microservices", "docker", "kubernetes", "caching", "message queues"],
    "keywords": ["api", "rest", "graphql", "scalable", "performance", "database", "server"],
    "technologies": ["python", "java", "node.js", "go", "postgresql", "mongodb", "redis", "kafka"]
  },
  "Frontend Developer": {
    "required_skills": ["html", "css", "javascript", "responsive design", "ui/ux"],
    "preferred_skills": ["react", "vue", "angular", "typescript", "testing", "performance"],
    "keywords": ["responsive", "user interface", "component", "ui", "ux", "interactive",
"accessibility"],
    "technologies": ["react", "vue", "angular", "typescript", "javascript", "css", "sass", "webpack"]
  },
  "Full Stack Developer": {
    "required_skills": ["frontend", "backend", "database", "api"],
    "preferred_skills": ["devops", "cloud", "system design", "testing"],
    "keywords": ["full stack", "end-to-end", "frontend", "backend", "database", "api", "deployment"],
    "technologies": ["react", "node.js", "python", "mongodb", "postgresql", "docker", "aws"]
  },
  "Data Scientist": {
    "required_skills": ["python", "machine learning", "statistics", "data analysis"],
    "preferred_skills": ["deep learning", "nlp", "computer vision", "big data"],
    "keywords": ["model", "prediction", "analysis", "machine learning", "data", "insights",
"algorithm"],
    "technologies": ["python", "tensorflow", "pytorch", "pandas", "numpy", "sql", "spark", "tableau"]
  },
  "Data Analyst": {
    "required_skills": ["sql", "excel", "data visualization", "statistics"],
    "preferred_skills": ["python", "tableau", "power bi", "data modeling"],
    "keywords": ["analysis", "insights", "dashboard", "report", "visualization", "kpi", "metrics"],
    "technologies": ["sql", "excel", "tableau", "power bi", "python", "r"]
  },
  "DevOps Engineer": {
    "required_skills": ["ci/cd", "docker", "kubernetes", "linux", "cloud"],
    "preferred_skills": ["terraform", "ansible", "monitoring", "security"],
    "keywords": ["automation", "deployment", "pipeline", "infrastructure", "monitoring",
"scalability"],
    "technologies": ["docker", "kubernetes", "jenkins", "aws", "gcp", "terraform", "ansible",
"prometheus"]
  },
  "ML Engineer": {
    "required_skills": ["machine learning", "python", "deep learning", "mlops"],
    "preferred_skills": ["distributed systems", "model deployment", "feature engineering"],
    "keywords": ["model", "training", "inference", "pipeline", "deployment", "optimization"],
    "technologies": ["python", "tensorflow", "pytorch", "mlflow", "kubeflow", "docker", "aws
sagemaker"]
  }
}

```

```

# Action verbs for strong resume bullets
ACTION_VERBS = {
    "high_impact": ["achieved", "accelerated", "delivered", "drove", "generated", "increased",
"launched", "led", "optimized", "reduced", "spearheaded", "transformed"],
    "technical": ["architected", "automated", "built", "coded", "debugged", "deployed", "designed",
"developed", "engineered", "implemented", "integrated", "migrated", "programmed", "refactored"],
    "collaboration": ["collaborated", "coordinated", "facilitated", "mentored", "partnered", "presented",
"trained"],
    "analysis": ["analyzed", "evaluated", "identified", "investigated", "researched", "solved", "tested",
"validated"]
}

# Common resume sections
RESUME_SECTIONS = ["education", "experience", "skills", "projects", "certifications",
"achievements", "summary", "objective"]

class ResumeAnalyzer:
    """Comprehensive AI-powered resume analyzer"""

    def __init__(self, model_service=None):
        """Initialize the analyzer with optional LLM service"""
        self.model_service = model_service
        self.version = "2.0"

    async def analyze_resume(
        self,
        resume_text: str,
        target_role: str = "Software Engineer",
        user_profile: Optional[Dict] = None
    ) -> Dict[str, Any]:
        """
        Perform comprehensive resume analysis

        Args:

```

Part B – Individual scoring methods that calculate specific aspects of the resume quality. The keyword score checks if important industry terms are present. The skills score matches technical competencies against role requirements. The action verb score checks for strong verbs like 'Developed', 'Implemented', 'Architected', and 'Optimized'.

```

def _calculate_keyword_score(self, resume_text: str, role_config: Dict) -> float:
    """Calculate keyword match score"""
    all_keywords = (
        role_config.get("required_skills", []) +
        role_config.get("preferred_skills", []) +
        role_config.get("keywords", []) +
        role_config.get("technologies", [])
    )

```

```

if not all_keywords:
    return 50.0

found = sum(1 for kw in all_keywords if kw.lower() in resume_text)
return min(100, (found / len(all_keywords)) * 150) # Scale up for partial matches

def _calculate_skills_score(self, skills: List[str], role_config: Dict) -> float:
    """Calculate skills match score"""
    required = role_config.get("required_skills", [])
    preferred = role_config.get("preferred_skills", [])
    technologies = role_config.get("technologies", [])

    skills_lower = [s.lower() for s in skills]

    required_found = sum(1 for s in required if any(s.lower() in skill for skill in skills_lower))
    preferred_found = sum(1 for s in preferred if any(s.lower() in skill for skill in skills_lower))
    tech_found = sum(1 for t in technologies if any(t.lower() in skill for skill in skills_lower))

    # Weight: required (50%), preferred (30%), technologies (20%)
    score = 0
    if required:
        score += (required_found / len(required)) * 50
    if preferred:
        score += (preferred_found / len(preferred)) * 30
    if technologies:
        score += (tech_found / len(technologies)) * 20

    return min(100, score)

def _analyze_format(self, resume_text: str) -> float:
    """Analyze resume format and structure"""
    score = 50 # Base score

    # Check for key sections
    for section in RESUME_SECTIONS:
        if section.lower() in resume_text.lower():
            score += 5

    # Check length (ideal: 500-2000 words)
    word_count = len(resume_text.split())
    if 500 <= word_count <= 2000:
        score += 10
    elif word_count < 300:
        score -= 15
    elif word_count > 3000:
        score -= 10

    # Check for contact info patterns
    if re.search(r'\b[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Z|a-z]{2,}\b', resume_text):
        score += 5
    if re.search(r'linkedin\.com|github\.com', resume_text, re.IGNORECASE):
        score += 5

```

```

    return min(100, max(0, score))

def _analyze_experience(self, resume_text: str, extracted_data: Dict) -> float:
    """Analyze work experience section"""
    score = 40 # Base score

    # Check for experience section
    if re.search(r'experience|work history|employment', resume_text, re.IGNORECASE):
        score += 15

    # Check for date ranges (indicates proper formatting)
    date_patterns = re.findall(r'\b(20\d{2}|19\d{2})\b', resume_text)
    if len(date_patterns) >= 2:
        score += 10

    # Check for company names followed by roles
    if re.search(r'(?:(?:at|@|,)\s*(?:Inc|LLC|Corp|Ltd|Company))', resume_text, re.IGNORECASE):
        score += 10

    # Check for bullet points (good formatting)
    bullet_count = len(re.findall(r'[•\-\*]\s', resume_text))
    if bullet_count >= 5:
        score += 15
    elif bullet_count >= 3:
        score += 10

    return min(100, score)

def _analyze_projects(self, projects: List[Dict], role_config: Dict) -> float:
    """Analyze projects section"""

```

4. Authentication & Token Management Middleware

This middleware handles all authentication and authorization in the platform. It implements a dual-token system using JSON Web Tokens (JWT). When a user logs in, two tokens are created: an access token (valid for 15 minutes) used for API requests, and a refresh token (valid for 7 days) used to obtain new access tokens without re-login. This is called token rotation. The system detects refresh token reuse – if someone tries to use an old refresh token, it indicates a potential security breach and all sessions for that user are immediately revoked. The middleware supports role-based access control with three levels: student, admin, and superadmin. Tokens are stored in secure HTTP-only cookies to prevent Cross-Site Scripting (XSS) attacks from stealing authentication credentials.

File: backend/src/middleware/auth.middleware.js

```

import jwt from 'jsonwebtoken';
import crypto from 'crypto';
import User from '../models/User.model.js';

```

```

import RefreshToken from '../models/RefreshToken.model.js';
import AuditLog from '../models/AuditLog.model.js';
import { securityConfig } from '../config/security.config.js';

/**
 * JWT Token Service
 * Handles access token and refresh token generation/validation
 */

/**
 * Generate Access Token
 * @param {Object} user - User object
 * @returns {string} JWT access token
 */
export const generateAccessToken = (user) => {
  const payload = {
    id: user._id || user.id,
    email: user.email,
    role: user.role || 'student',
    type: 'access',
  };

  return jwt.sign(payload, process.env.JWT_SECRET, {
    expiresIn: securityConfig.jwt.accessTokenExpiry,
    issuer: securityConfig.jwt.issuer,
    audience: securityConfig.jwt.audience,
    algorithm: securityConfig.jwt.algorithm,
  });
};

/**
 * Generate Refresh Token
 * @param {Object} user - User object
 * @param {string} tokenFamily - Token family for rotation tracking
 * @param {Object} requestInfo - Request information for logging
 * @returns {Object} Refresh token data
 */
export const generateRefreshToken = async (user, tokenFamily = null, requestInfo = {}) => {
  // Generate token family if not provided (new login)
  const family = tokenFamily || crypto.randomBytes(16).toString('hex');

  const payload = {
    id: user._id || user.id,
    family,
    type: 'refresh',
  };

  const token = jwt.sign(payload, process.env.JWT_REFRESH_SECRET ||
process.env.JWT_SECRET, {
    expiresIn: securityConfig.jwt.refreshTokenExpiry,
    issuer: securityConfig.jwt.issuer,
    audience: securityConfig.jwt.audience,

```

```

});

// Calculate expiry date
const expiresAt = new Date(Date.now() + 7 * 24 * 60 * 60 * 1000); // 7 days

// Parse user agent for device info
const deviceInfo = parseUserAgent(requestInfo.userAgent || "");

// Store refresh token in database
const refreshTokenDoc = await RefreshToken.create({
  userId: user._id || user.id,
  token: hashToken(token),
  tokenFamily: family,
  expiresAt,
  createdByIp: requestInfo.ip || "",
  userAgent: requestInfo.userAgent || "",
  deviceInfo,
});

return {
  token,
  expiresAt,
  tokenId: refreshTokenDoc._id,
  tokenFamily: family,
};
};

/**
 * Hash token for secure storage
 */
const hashToken = (token) => {
  return crypto
    .createHash('sha256')
    .update(token)
    .digest('hex');
};

/**
 * Parse user agent string
 */
const parseUserAgent = (userAgent) => {
  const result = {
    browser: 'Unknown',
    os: 'Unknown',
    device: 'Unknown',
  };

  if (!userAgent) return result;

  // Simple parsing
  if (userAgent.includes('Chrome')) result.browser = 'Chrome';
  else if (userAgent.includes('Firefox')) result.browser = 'Firefox';

```

```

else if (userAgent.includes('Safari')) result.browser = 'Safari';
else if (userAgent.includes('Edge')) result.browser = 'Edge';

if (userAgent.includes('Windows')) result.os = 'Windows';
else if (userAgent.includes('Mac')) result.os = 'MacOS';
else if (userAgent.includes('Linux')) result.os = 'Linux';
else if (userAgent.includes('Android')) result.os = 'Android';
else if (userAgent.includes('iOS') || userAgent.includes('iPhone')) result.os = 'iOS';

if (userAgent.includes('Mobile')) result.device = 'Mobile';
else if (userAgent.includes('Tablet')) result.device = 'Tablet';
else result.device = 'Desktop';

return result;

```

5. Job Matching & Market Intelligence Engine

The Job Matcher calculates how well a student's profile aligns with available job openings. It uses a weighted scoring algorithm where skills match contributes 50%, experience match 30%, and education/location match 20%. The service includes an intelligent skill normalization system that treats equivalent terms as the same skill – for example, 'JS', 'JavaScript', and 'ECMAScript' are all recognized as the same technology. The synonym mapping covers over 100 common abbreviations and alternative names. The output includes the overall match percentage, a categorized list of matched and missing skills, salary range estimates, and personalized recommendations for improving the match score.

File: ai-service/app/services/job_matcher.py

```

"""
AI Job Matcher Service
Provides intelligent job matching based on user profiles
"""

from typing import List, Dict, Optional, Any
import json
import asyncio
from app.services.model_service import ModelService
from app.services.embedding_service import EmbeddingService
import logging

logger = logging.getLogger(__name__)

class JobMatcherService:
    """AI-powered job matching and recommendation engine"""

    def __init__(self):
        self.model_service = ModelService()
        self.embedding_service = EmbeddingService()

```

```

# Skill synonyms for better matching
self.skill_synonyms = {
    "javascript": ["js", "ecmascript", "es6", "es2015"],
    "typescript": ["ts"],
    "python": ["py", "python3"],
    "react": ["reactjs", "react.js"],
    "node": ["nodejs", "node.js"],
    "vue": ["vuejs", "vue.js"],
    "angular": ["angularjs", "angular.js"],
    "ml": ["machine learning", "machinelearning"],
    "ai": ["artificial intelligence"],
    "aws": ["amazon web services"],
    "gcp": ["google cloud", "google cloud platform"],
    "azure": ["microsoft azure"],
    "sql": ["mysql", "postgresql", "postgres", "sqlite"],
    "nosql": ["mongodb", "mongo", "dynamodb", "cassandra"],
    "docker": ["containerization", "containers"],
    "k8s": ["kubernetes"],
    "ci/cd": ["cicd", "continuous integration", "continuous deployment"],
}

def _normalize_skill(self, skill: str) -> str:
    """Normalize skill name for comparison"""
    skill_lower = skill.lower().strip()

    # Check if skill is a synonym
    for main_skill, synonyms in self.skill_synonyms.items():
        if skill_lower in synonyms or skill_lower == main_skill:
            return main_skill

    return skill_lower

def _calculate_skill_match(
    self,
    user_skills: List[str],
    job_skills: List[str]
) -> Dict[str, Any]:
    """Calculate skill match score and details"""
    user_skills_normalized = set(self._normalize_skill(s) for s in user_skills)
    job_skills_normalized = set(self._normalize_skill(s) for s in job_skills)

    if not job_skills_normalized:
        return {
            "score": 50,
            "matched": [],
            "missing": [],
            "extra": list(user_skills_normalized)
        }

    matched = user_skills_normalized.intersection(job_skills_normalized)
    missing = job_skills_normalized - user_skills_normalized

```



```

extra = user_skills_normalized - job_skills_normalized

# Score calculation: matched skills / required skills * 100
score = int((len(matched) / len(job_skills_normalized)) * 100)

return {
    "score": min(100, score),
    "matched": list(matched),
    "missing": list(missing),
    "extra": list(extra)
}

def _calculate_experience_match(
    self,
    user_experience_years: float,
    job_experience_min: float,
    job_experience_max: Optional[float] = None
) -> Dict[str, Any]:
    """Calculate experience match score"""
    if job_experience_min <= 0:
        return {"score": 100, "status": "entry_level"}

    if user_experience_years >= job_experience_min:
        if job_experience_max and user_experience_years > job_experience_max:
            return {"score": 80, "status": "overqualified"}
        return {"score": 100, "status": "qualified"}

```

6. Proctoring & Assessment Integrity System

The Proctoring System ensures fair and honest assessments by monitoring student behavior during online tests in real-time. It uses the device camera and browser APIs to detect violations such as: tab switching (student navigating away from the test), missing face detection (student not visible on camera), multiple faces detected (someone helping the student), and camera being covered or obstructed. Each violation type has a different severity level and point deduction. Critical violations like no face detected reduce the integrity score by 15 points, while minor warnings like a brief tab switch reduce it by 5 points. The system uses MongoDB TTL (Time To Live) indexes to automatically delete proctoring session data after 90 days for privacy compliance with data protection regulations. If a student accumulates too many violations, the assessment is automatically terminated and flagged for manual review by an administrator.

File: backend/src/models/ProctoringSession.model.js

```

/**
 * Proctoring Session Model
 * Secure storage for proctoring data with auto-delete after retention period
 */

import mongoose from 'mongoose';
import { encrypt, decrypt } from '../utils/encryption.js';

```

```
const RETENTION_DAYS = 30; // Auto-delete after 30 days
```

```
const ViolationSchema = new mongoose.Schema({
  type: {
    type: String,
    required: true,
    enum: [
      'tab_switch',
      'fullscreen_exit',
      'keyboard_shortcut',
      'right_click',
      'copy_paste',
      'no_face',
      'multiple_faces',
      'camera_covered',
      'camera_inactive',
      'background_noise',
      'screen_share_stopped',
      'suspicious_behavior',
      'device_change',
      'network_issue',
    ],
  },
  timestamp: {
    type: Date,
    default: Date.now,
  },
  description: {
    type: String,
    required: true,
  },
  severity: {
    type: String,
    enum: ['warning', 'critical'],
    default: 'warning',
  },
  screenshot: {
    type: String, // Encrypted base64 screenshot
    select: false, // Don't include by default
  },
});
```

```
const ProctoringSessionSchema = new mongoose.Schema({
  // Reference to test session
  testSession: {
    type: mongoose.Schema.Types.ObjectId,
    ref: 'TestSession',
    required: true,
    index: true,
  },
});
```

```

// Reference to user
user: {
  type: mongoose.Schema.Types.ObjectId,
  ref: 'User',
  required: true,
  index: true,
},

// Consent tracking
consent: {
  given: {
    type: Boolean,
    default: false,
  },
  timestamp: Date,
  ipAddress: String,
  userAgent: String,
},

// Session timing
startTime: {
  type: Date,
  required: true,
  default: Date.now,
},
endTime: Date,
duration: Number, // in seconds

// Proctoring status
status: {
  type: String,
  enum: ['active', 'completed', 'terminated', 'paused'],
  default: 'active',
  index: true,
},

// Termination reason
terminationReason: String,

// Device information (encrypted)
deviceInfo: {

```

7. AI Recommendation Engine

The Recommendation Engine is responsible for generating personalized career guidance for each student. It analyzes the student's current skill set, assessment results, resume analysis, and career goals to create tailored learning paths. The engine uses vector similarity matching to find the most relevant resources from a curated database of courses, certifications, and tutorials. It prioritizes recommendations based on skill gap severity – skills where the student scored lowest receive the highest priority. The engine also

considers the student's learning pace and preferred learning style to suggest appropriate difficulty levels. Each recommendation includes an estimated time to completion, relevance score, and expected impact on the student's overall career readiness.

File: ai-service/app/services/recommendation_engine.py

```
"""
Prepzo AI Service - Recommendation Engine
Fully AI-driven recommendation system with no hardcoded mappings
"""

from loguru import logger # pyre-ignore
from typing import List, Dict, Any, Optional, Tuple
import numpy as np # pyre-ignore
import hashlib
import asyncio
import copy
import re
import json
from datetime import datetime

from app.services.embedding_service import EmbeddingService # pyre-ignore
from app.services.vector_store import VectorStore # pyre-ignore
from app.services.model_service import ModelService # pyre-ignore


class RecommendationEngine:
    """
    AI-Powered Recommendation Engine

    NO hardcoded mappings. All recommendations are determined by:
    - Vector similarity between student profile and resources
    - Gap magnitude weighting
    - Industry demand factors
    - Historical effectiveness scores
    - Student level matching
    """

    def __init__(
        self,
        embedding_service: EmbeddingService,
        vector_store: VectorStore,
        model_service: ModelService,
        database
    ):
        self.embedding_service = embedding_service
        self.vector_store = vector_store
        self.model_service = model_service
        self.db = database

    def _parse_json(raw: str) -> Optional[Dict[str, Any]]:
        """Robustly extract first JSON object from model output."""
```

```

if not raw:
    return None
for pattern in (
    r"```(?:json)?\s*([\s\S]*?)\s*```",
    r"(\{[\s\S]*\})",
):
    m = re.search(pattern, raw)
    if m:
        candidate = m.group(1)
        try:
            return json.loads(candidate)
        except:
            continue
    try:
        return json.loads(raw)
    except:
        return None

async def generate_recommendations(
    self,
    student_profile: Dict[str, Any],
    assessment_results: Dict[str, Any],
    include_reasoning: bool = True
) -> Dict[str, Any]:
    """
    Generate fully dynamic AI-powered recommendations

    Args:
        student_profile: Complete student profile
        assessment_results: Latest assessment results
        include_reasoning: Whether to include AI reasoning

    Returns:
        Comprehensive recommendation package
    """
    logger.info(f"Generating recommendations for student {student_profile.get('_id')}")

    # Check for low-score/skip-all scenario
    # Map both camelCase and snake_case for robustness
    # Deepcopy to prevent Pydantic v2 "Already borrowed" errors
    assessment_results = copy.deepcopy(assessment_results)
    student_profile = copy.deepcopy(student_profile)

    score_val = assessment_results.get('overallScore', assessment_results.get('overall_score',
    assessment_results.get('score', 0)))

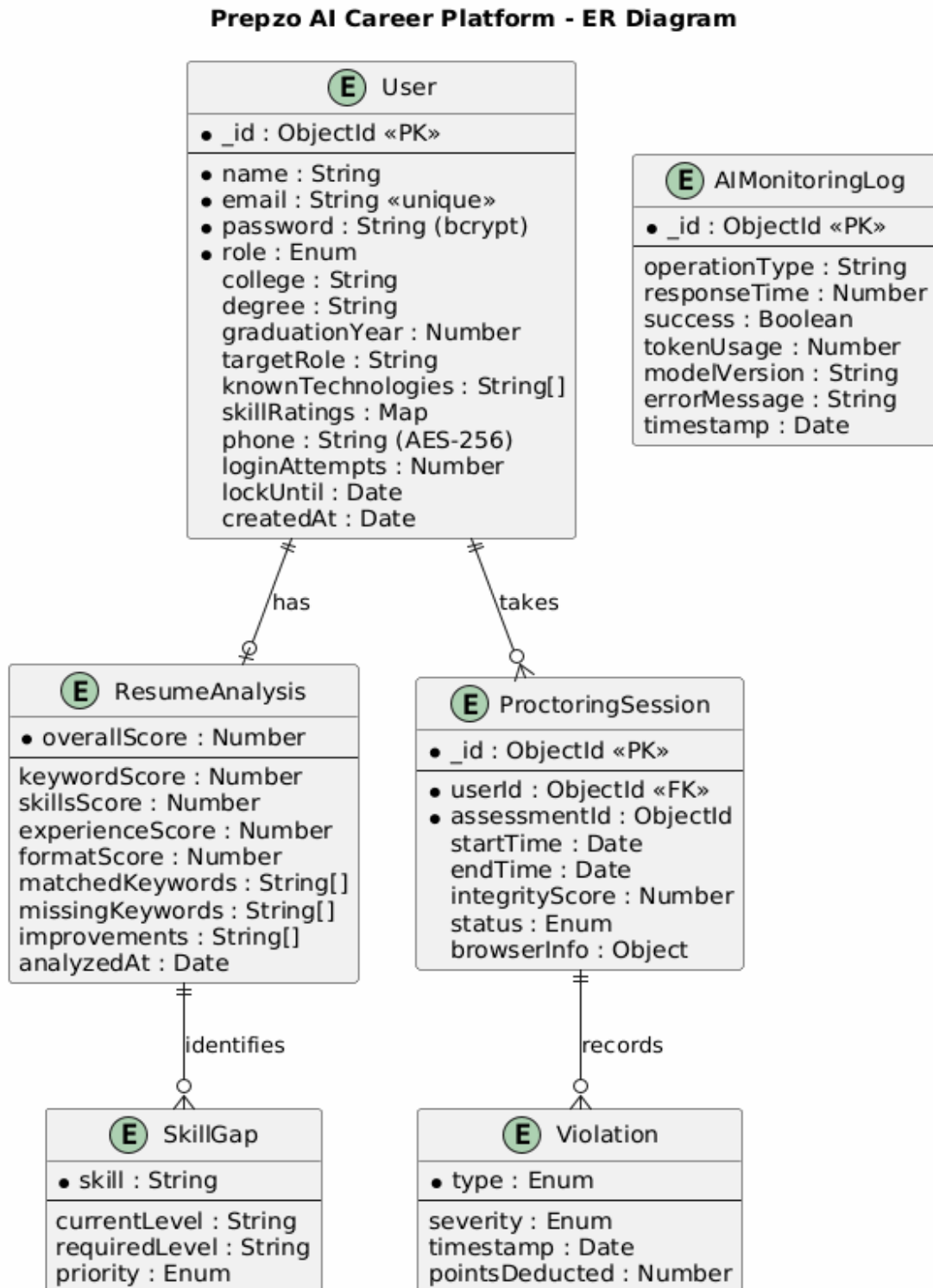
    try:
        current_score = float(str(score_val).replace('%', '').strip())
    except (ValueError, TypeError):
        current_score = 0

    is_low_score = current_score <= 20

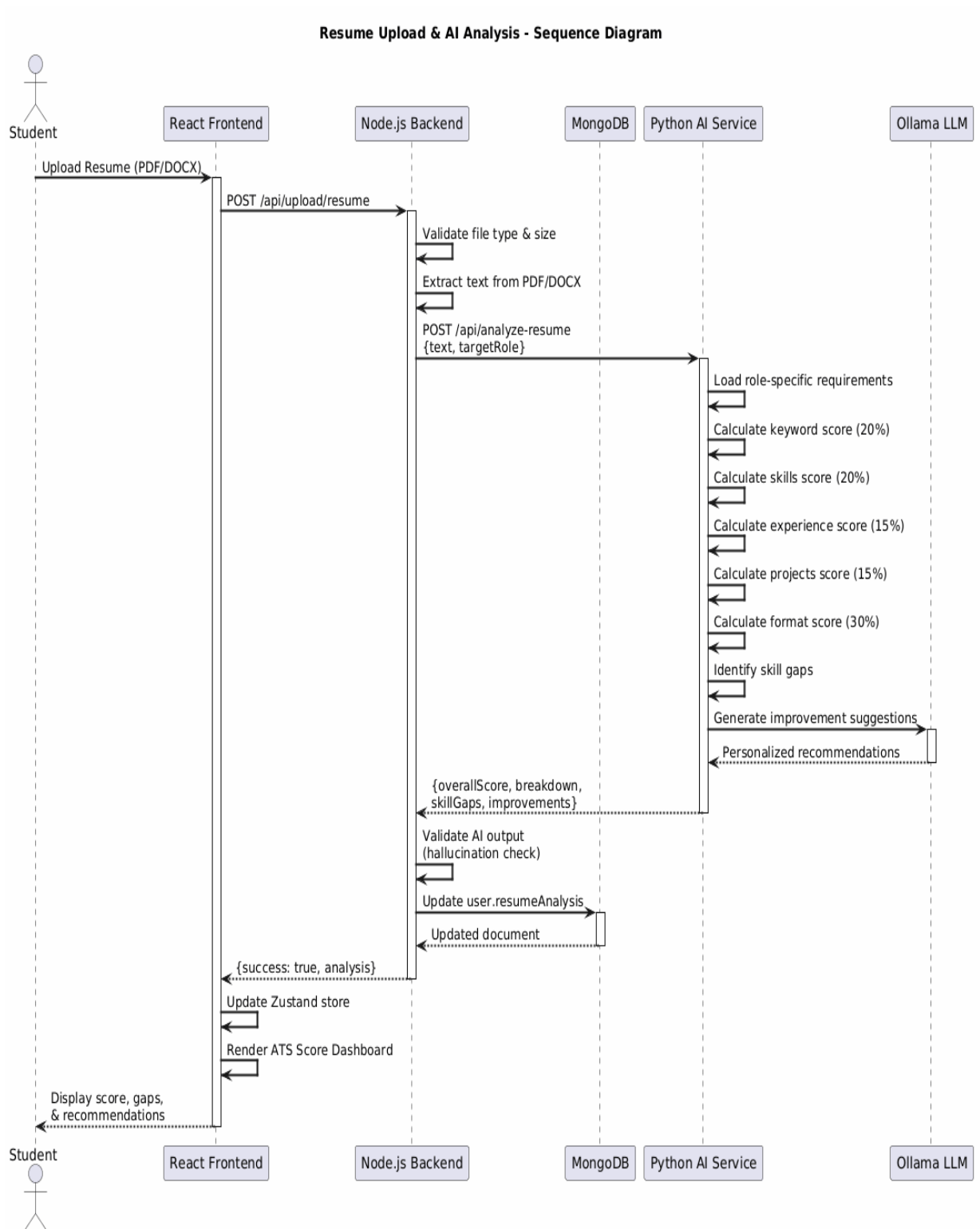
```

Diagrams

1. ER Diagram



2. Sequence Diagram



3. Class Diagram

